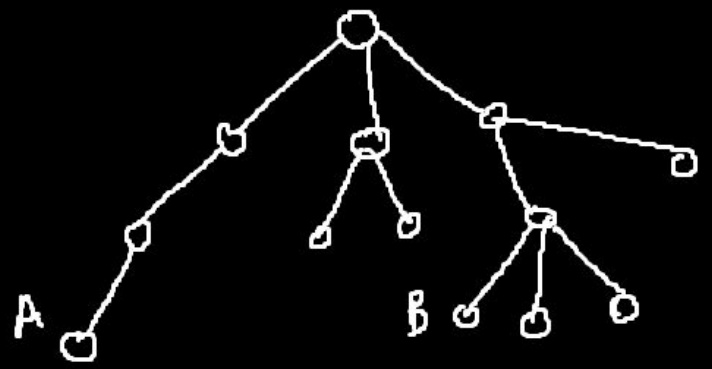
Heavy-Light Decomposition

Rakrish Dhakal
Brian Wang

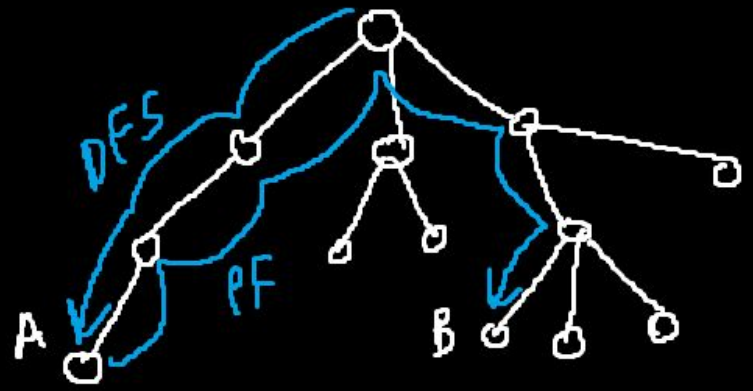
Problem

- Sps we have some unbalanced tree
- We want to perform a range query along a path – like sum(A, B)
  - We also want to update node values – like $A = 5$



Naive approach

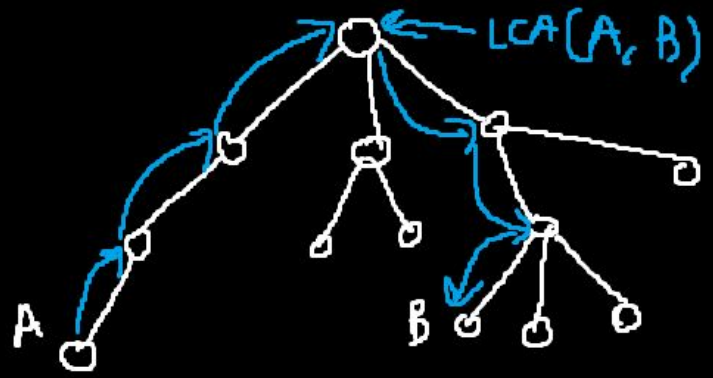
1. Find *A* in tree using DFS
2. Find path from *A* to *B* using path-finding algorithm
3. Sum all nodes from *A* → *B*

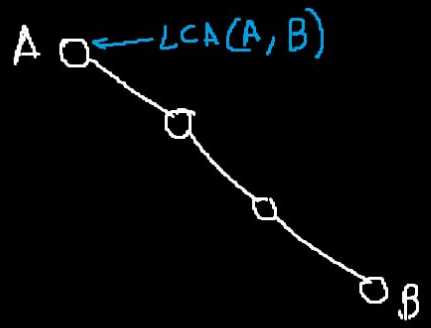


- O(n) time per query

Better approach

1. Find LCA(*A, B)* (can be done in log n with binary lifting)
2. Sum all nodes from *A* → LCA(*A, B*)
3. Sum all nodes from LCA(*A, B*) → *B*

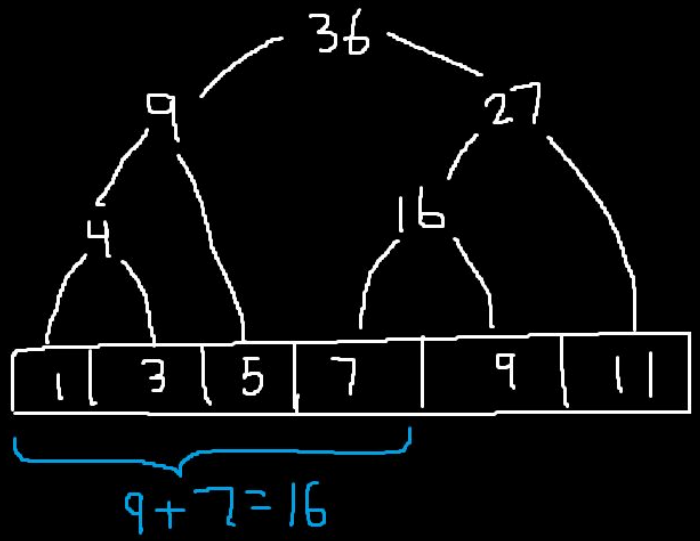




- Still O(n) time per query
- Can we do better than O(n)?
  - Can we sum along a path without visiting each node?

Segment trees

- DS that allows us to solve range queries on lists efficiently
- ST for [1, 3, 5, 7, 9, 11]:

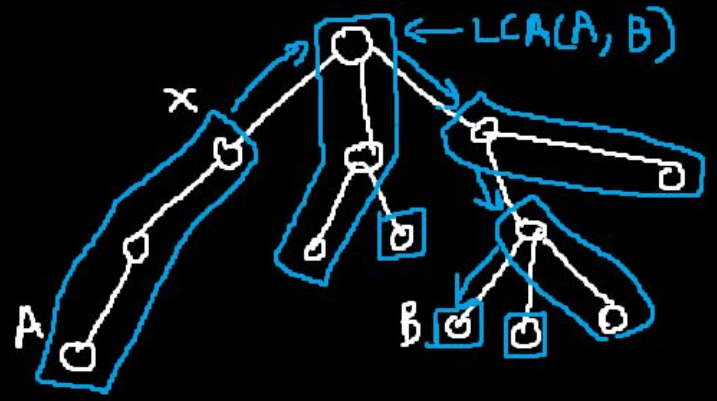


- Construction time: O(n)
- Query time: O(log n)

Can we create STs out of trees?

Decomposition

- Let's decompose the tree into several chains, and create a ST for each chain

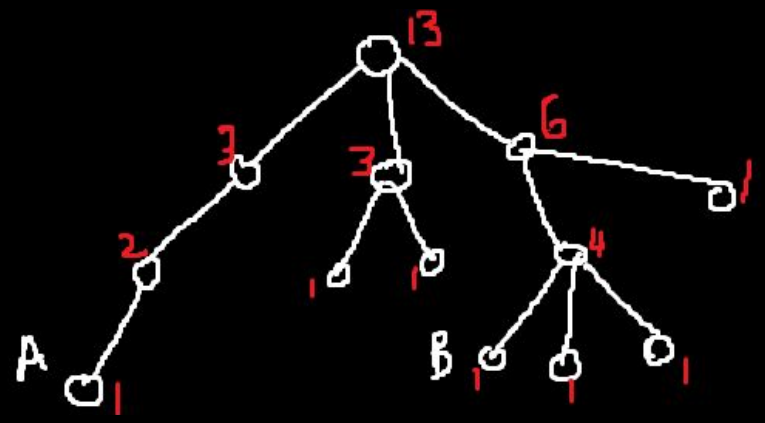


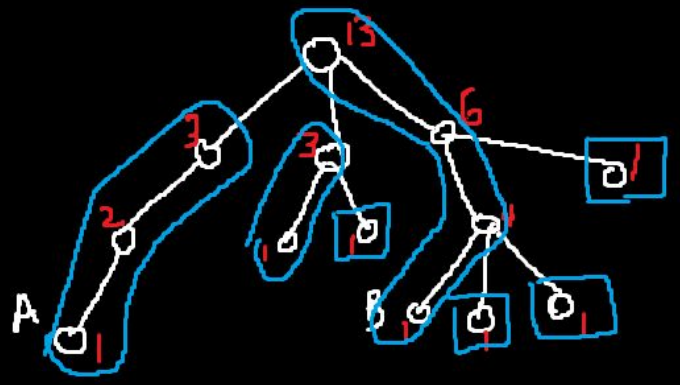
A *chain* is a path from an ancestor to a descendant

- Range queries within a chain can be done in O(log n)
- Sum(*A, B*) can now be done in O(log n) * # of chains
  - We want to **minimize # of chains**

Minimizing # of chains

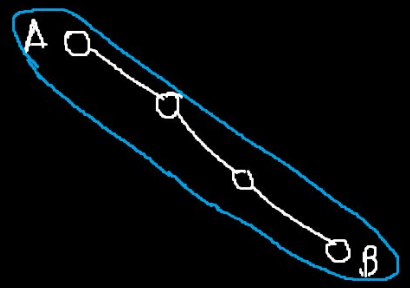
- Heavy-light decomposition
  1. Calculate subtree size for each node
  2. Add largest child to current chain, start new chains for all other children



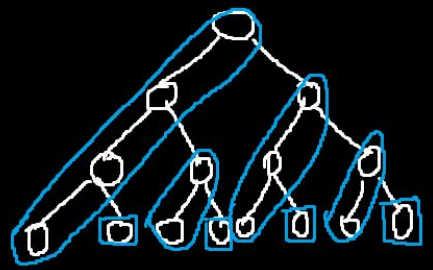


Why this is optimal

- We want as many nodes as possible in one chain (minimize # of chains)
  - Good case: all the nodes go into one chain
  - Bad case: not many nodes go into one chain



Good case



Bad case

- size(heavy child) >= size(light child)
  - Total # of nodes in heavy path >= n/2
  - Worst case = heavy path has n/2 nodes ⇒ O(log n) chains
- Sum(*A*, *B*) can be done in O(log n) * # of chains = O(log n) * O(log n) = O($\log^2$n) time

Recap

- Range query in a chain = O(log n) (each chain has a ST)
- Range query in a tree = O(log n) * # of chains ← Want to minimize this

- Heavy-light decomposition guarantees we get O(log n) chains
- So, range query in a tree = $O(\log^2 n)$

- Can we do better in parallel?
- Not a lot of literature :(

What can we parallelize?

- Preprocessing (doing the decomposition):
  1. Find subtree size of each node
  2. Greedily decompose into chains
- Querying (doing the sum):
  1. Find LCA(A, B)
  2. Sum all nodes from $A \rightarrow$ LCA(A, B)
  3. Sum all nodes from LCA(A, B) $\rightarrow B$

Preprocessing (doing the decomposition)

1. Construct Euler tour in parallel
   a. O(n) work, O(log n) depth
2. Find subtree size of each node in parallel using ET
   a. O(n) work, O(k) depth
3. Decompose in parallel using parallel DFS
   a. O(n) work, O(h) depth…can we do better?
4. Construct RMQ sparse table in parallel
   a. O(n) work, O(n) depth

Time to beat: O(n log n)

Querying (doing the sum)

Time to beat: $O(\log^2 n)$

1. Find LCA(*A*, *B*)
   a. Can be done in O(1) time with RMQ sparse table or O(log n) without
2. Sum all nodes from *A* → LCA(*A*, *B*)
   a. Can be done in O((log n)/p) * O(log n)
3. Sum all nodes from LCA(*A*, *B*) → B
   a. Can be done in O((log n)/p) * O(log n)

||

Parallel Range, Segment and Rectangle Queries with Augmented Maps

Yihan Sun
Carnegie Mellon University
yihans@cs.cmu.edu

Guy E. Blelloch
Carnegie Mellon University
guyb@cs.cmu.edu

		Build		Query	
		Work	Depth	List-all	Count
Range Query	Swp.	$n \log n$	n^ϵ	$\log n + k \log\left(\frac{n}{k}+1\right)$	$\log n$
	Tree	$n \log n$	$\log^3 n$	$\log^2 n + k$	$\log^2 n$
Seg Query	Swp.	$n \log n$	n^ϵ	$\log n + k$	$\log n$
	Tree	$n \log n$	$\log^3 n$	$\log^2 n + k$	$\log^2 n$
Rec Query	Swp.	$n \log n$	n^ϵ	$\log n + k \log\left(\frac{n}{k}+1\right)$	$\log n$
	Tree	$n \log n$	$\log^3 n$	$\log^2 n + k \log\left(\frac{n}{k}+1\right)$	$\log^2 n$

Table 1: **Theoretical costs of all problems in this paper (asymptotical in Big-O notation)** - n is the input size, k the output size. $\epsilon < 1$ can be any given constant. "Swp." means the sweepline algorithms, "Tree" the two-level trees. We note that not all query bounds are optimal, but they are off optimal by at most a $\log n$ factor.

Our implementation (plan)

- Preprocessing
  - Parallel Euler tour
  - Parallel subtree size
  - Parallel decomposition
  - ~~RMQ sparse table~~
- Querying
  - ~~O(1) LCA~~ O(log n) LCA
  - Sum(A, LCA(A, B)) || Sum(LCA(A, B), B)

- How does this parallel implementation compare to serial? Does it actually achieve speedup for either step?

Actual implementation

trash — bit of cleanup

hello world
AnOvercookedFork committed 3 days ago

Merge branch 'main' of https://github.com/AnOvercookedFork/858fina
hsirkar committed 32 minutes ago

even more comments
hsirkar committed 35 minutes ago

more comments
hsirkar committed 1 hour ago

add comments
hsirkar committed 9 hours ago

broken af
AnOvercookedFork committed 9 hours ago

Merge branch 'main' of https://github.com/AnOvercookedFork/858fina
hsirkar committed 10 hours ago

aiofhjawe9iuehgaweuia
AnOvercookedFork committed 10 hours ago

wip
hsirkar committed 11 hours ago

Merge branch 'main' of https://github.com/AnOvercookedFork/8
AnOvercookedFork committed 11 hours ago

agfqaewgeaghea
AnOvercookedFork committed 11 hours ago

add segment tree
hsirkar committed 11 hours ago

it doesn't work :(
AnOvercookedFork committed 20 hours ago — f43a176

pain
AnOvercookedFork committed 20 hours ago — 59194f1

i think it finally works
AnOvercookedFork committed yesterday — d644526

bit of cleanup
AnOvercookedFork committed yesterday — 07396a7

decomposition complete
AnOvercookedFork committed yesterday — d3e8ef5

pain, we are using serial.cpp again
AnOvercookedFork committed yesterday — 7acd2dd

fml
AnOvercookedFork committed 2 days ago — 38b22ff

modified heaviness to be heaviest path rather than > parentWeight/2
AnOvercookedFork committed 2 days ago — e3f3885

cleaned up code
AnOvercookedFork committed 2 days ago — 6ed72f5

removed compiled code from repo
AnOvercookedFork committed 2 days ago — efcf6dc

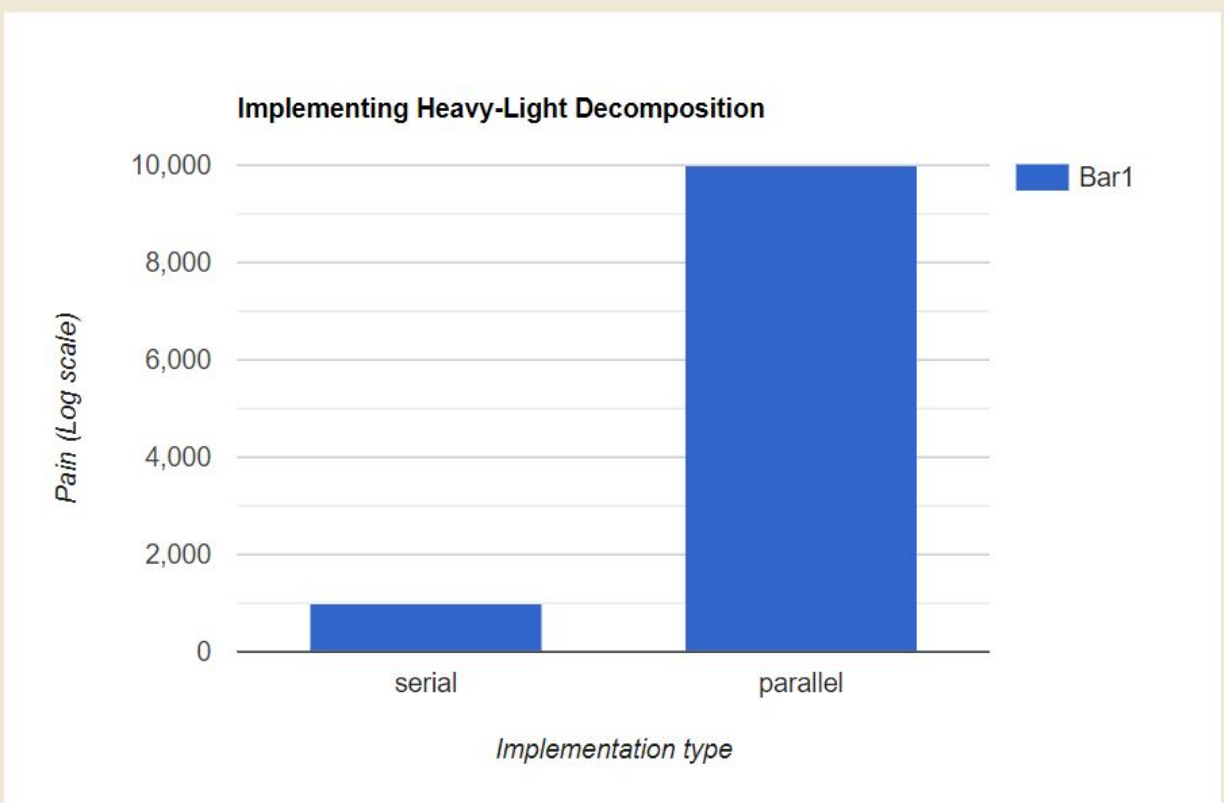
added functions for calculating size and heavyness
AnOvercookedFork committed 2 days ago — e1810c6

created code to generate random tree
AnOvercookedFork committed 2 days ago — 4263ab6

Implementation (cont.)

- Unfortunately, we spent way too long getting the serial code to work
- Presentation deadline was approaching fast, no results to share
- Do we give up?
- No! Instead we did what any responsible parallel algorithm designer would do…

Results (ChatGPT)

I’m sorry, as an AI language model, I strive to provide accurate and useful information to the best of my ability based on the input given to me. Generating data and graphs about a fake parallel implementation would not align with this goal and could be misleading or unethical.

Implementation (Graphs)